

AI POWERED RETAIL CUSTOMER INTELLIGENCE PLATFORM

PART A: Dataset Understanding

1. Identify the domain of the dataset.

The selected dataset belongs to the Retail .It contains customer information such as age , income, spending habits,purchase frequency,browsing behavior, and customer segments. This data can be used for customer analysis, segmentation and marketing strategies.

2. List all numerical and categorical attributes.

```
customer_id      int64
age              int64
annual_income    float64
months_active    int64
avg_monthly_spend float64
purchase_frequency float64
avg_order_value  float64
discount_usage_rate float64
return_rate      float64
browsing_time_minutes float64
support_interactions float64
payment_method   object
region           object
customer_segment object
dtype: object
```

3. Explain why normalization is required for the selected dataset

```
print(df.describe())
```

	customer_id	age	annual_income	months_active
count	50000.000000	50000.000000	46925.000000	50000.000000
mean	25000.500000	44.002820	44542.620904	36.477480
std	14433.901067	15.269871	30537.946035	20.787197
min	1.000000	18.000000	2946.317892	1.000000
25%	12500.750000	31.000000	24373.244554	18.000000
50%	25000.500000	44.000000	36787.730010	36.000000
75%	37500.250000	57.000000	55497.245522	55.000000
max	50000.000000	70.000000	530788.317921	72.000000

	avg_monthly_spend	purchase_frequency	avg_order_value
count	47480.000000	48021.000000	50000.000000
mean	337.294238	4.910841	77.375915
std	204.627095	3.416115	70.531793
min	8.072298	0.017013	0.940393
25%	192.074544	2.456867	32.018468
50%	297.477404	4.158673	56.600351
75%	439.112361	6.526903	98.832037
max	3026.342238	67.682528	1289.427328

2648547

...	discount_usage_rate	return_rate	browsing_time_minutes	\
count	47451.000000	47513.000000	46066.000000	
mean	0.284358	0.148964	61.781329	
std	0.159236	0.110508	41.743394	
min	0.000534	0.000050	0.236109	
25%	0.160497	0.063938	31.082283	
50%	0.262591	0.123628	52.679175	
75%	0.387640	0.209612	82.549571	
max	0.934655	0.830395	406.813006	
	support_interactions			
count	48012.000000			
mean	1.494606			
std	1.221280			
min	0.000000			
25%	1.000000			
50%	1.000000			
75%	2.000000			
max	9.000000			

Normalization is required because the dataset contains numerical values with different ranges. Features with larger values can influence the analysis more than smaller values. Normalization scales all numerical features to be similar range, improving accuracy, comparison and the performance of machine learning models

PART B:Data Preparation

1. Load and inspect the dataset

```
from google.colab import files
uploaded = files.upload()

... Choose Files retail_custo...entation.csv
retail_customer_segmentation.csv(text/csv) - 8135306 bytes, last modified: 6/16/2026 - 100% done
Saving retail_customer_segmentation.csv to retail_customer_segmentation.csv
```

```
import pandas as pd

df = pd.read_csv('retail_customer_segmentation.csv')
print(df.head())
```

	customer_id	age	annual_income	months_active	avg_monthly_spend	\
0	33554	53	100473.211709	63	121.430565	
1	9428	54	54730.644845	67	572.552674	
2	200	44	58268.121079	57	266.593896	
3	12448	54	64829.795654	40	691.452358	
4	39490	28	27431.467873	15	832.664792	

2648547

	purchase_frequency	avg_order_value	discount_usage_rate	return_rate	\
0	0.817268	66.820403	0.117256	0.023144	
1	3.176551	137.087449	0.261647	0.429054	
2	2.713168	71.796888	0.284785	0.011854	
3	5.553977	105.501185	0.104832	0.399686	
4	1.348389	354.568534	0.409204	0.039517	

	browsing_time_minutes	support_interactions	payment_method	region	\
0	77.298393	2.0	Card	Semi-Urban	
1	92.132565	2.0	Wallet	Urban	
2	155.194768	1.0	UPI	Rural	
3	113.917756	0.0	Wallet	Rural	
4	50.123656	1.0	Card	Semi-Urban	

	customer_segment
0	Occasional
1	Occasional
2	Occasional
3	High_Value
4	Occasional

```
print(df.shape)
```

```
(50000, 14)
```

2. Handle missing values(if any)

```
print(df.isnull().sum())
```

```
customer_id      0
age              0
annual_income    3075
months_active    0
avg_monthly_spend 2520
purchase_frequency 1979
avg_order_value  0
discount_usage_rate 2549
return_rate      2487
browsing_time_minutes 3934
support_interactions 1988
payment_method    0
region            0
customer_segment  0
dtype: int64
```

```
df.fillna(df.median(numeric_only=True), inplace=True)

print(df.isnull().sum())
```

customer_id	0
age	0
annual_income	0
months_active	0
avg_monthly_spend	0
purchase_frequency	0
avg_order_value	0
discount_usage_rate	0
return_rate	0
browsing_time_minutes	0
support_interactions	0
payment_method	0
region	0
customer_segment	0
dtype: int64	

3. Remove duplicate records(if any)

```
print("Duplicate records before removal:", df.duplicated().sum())

Duplicate records before removal: 0

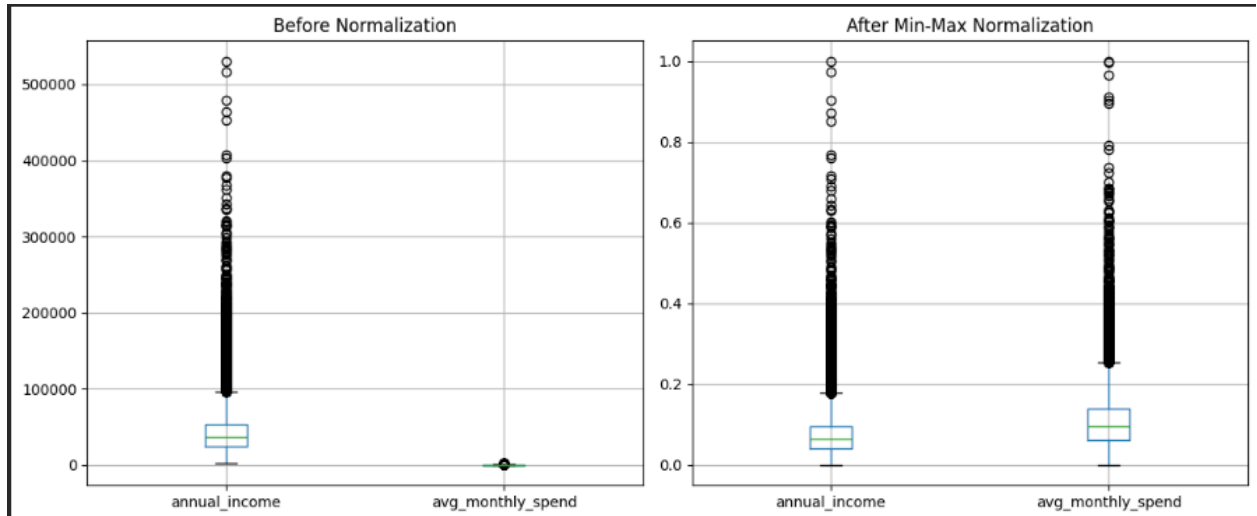
df.drop_duplicates(inplace=True)
print("Duplicate Records after removal:", df.duplicated().sum())

Duplicate Records after removal: 0
```

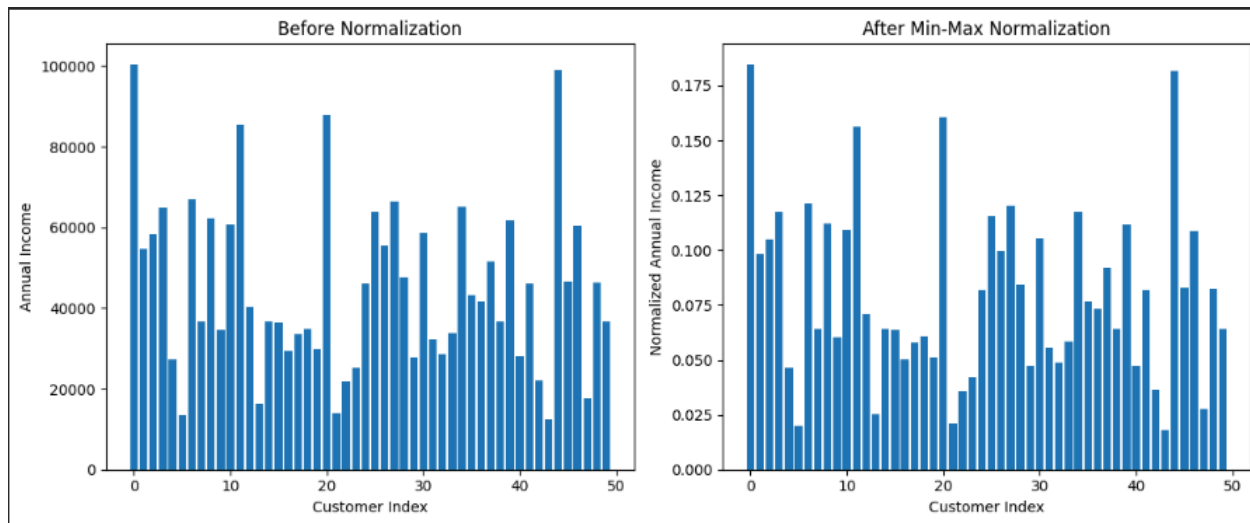
4. Add Proper visualization before and after normalization(Bar graph,Box plot,Line graph....)

Min Max Normalization:

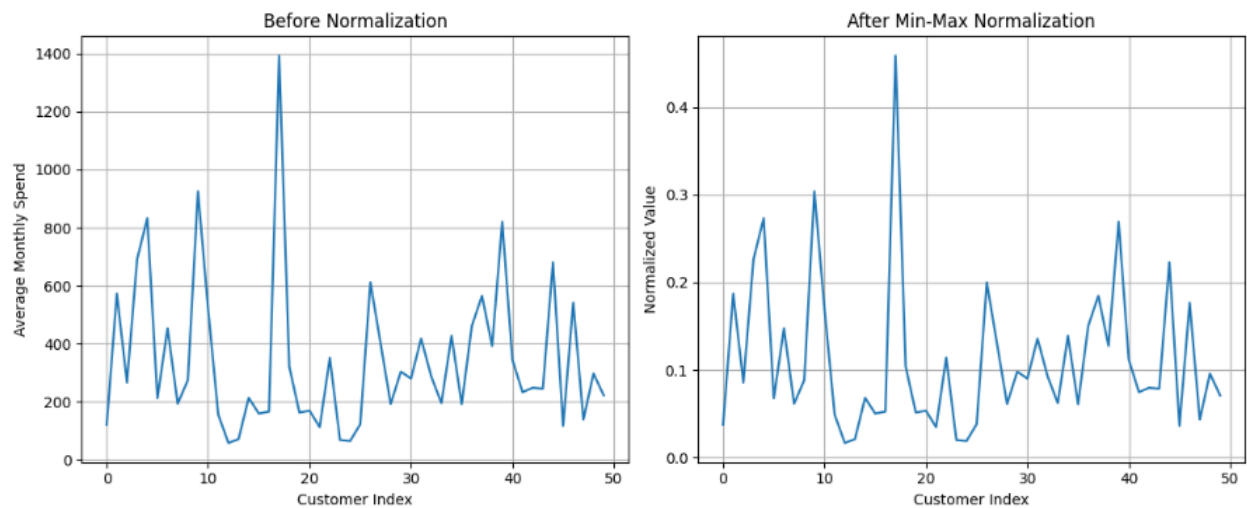
- Box Plot(Annual Income, avg_monthly spend)



- Bar Graph(Annual Income)



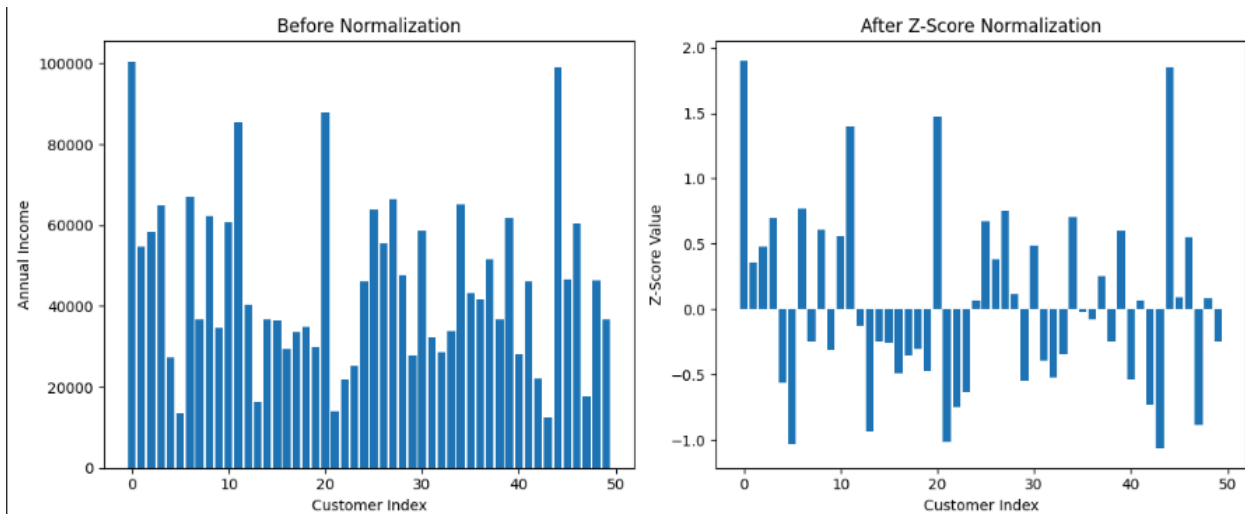
- Line Graph(Avg_monthly_spend)



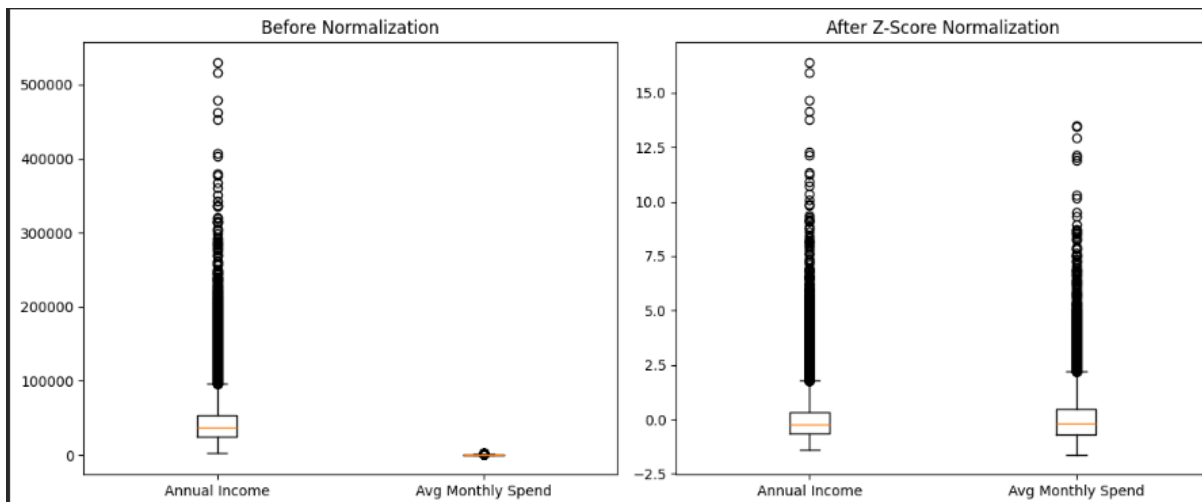
2648547

Z-Score Normalization

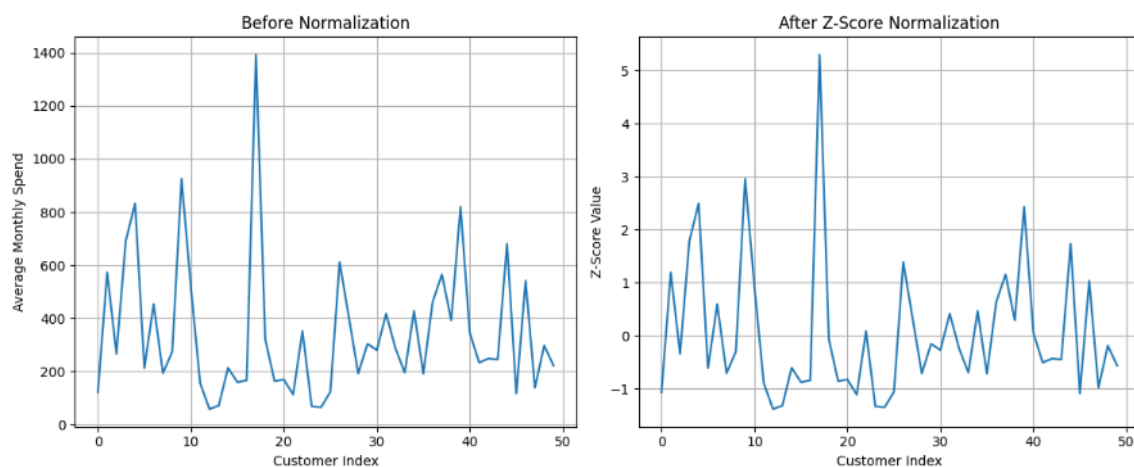
- Bar Graph(Annual Income)



- Box plot(Annual Income,Avg_monthly_spend)



- Line Graph(Avg_monthly_spend)



2648547

PART C: Normalization Techniques

Apply the following normalization methods to suitable numerical attributes

1. Min-Max Normalization

- Scale the values between 0 and 1
- Compare original and normalized values

```
})
print(comparison.head(10))
```

	Original_Income	Normalized_Income
0	100473.211709	0.184765
1	54730.644845	0.098106
2	58268.121079	0.104808
3	64829.795654	0.117239
4	27431.467873	0.046387
5	13511.061644	0.020015
6	66876.743284	0.121117
7	36787.730010	0.064113
8	62195.525423	0.112248
9	34685.254336	0.060130

2. Z-score Normalization(Standardization)

- Transform data to have mean=0 and standard deviation=1.
- Analyzing the resulting distribution

	age	annual_income	months_active	avg_monthly_spend	\
0	0.589217	1.902940	1.275919	-1.071471	
1	0.654706	0.359788	1.468347	1.188752	
2	-0.000185	0.479127	0.987277	-0.344170	
3	0.654706	0.700489	0.169458	1.784466	
4	-1.048010	-0.561165	-1.033217	2.491972	

	purchase_frequency	avg_order_value	discount_usage_rate	return_rate	\
0	-1.212712	-0.149658	-1.069563	-1.154784	
1	-0.508660	0.846599	-0.139189	2.608367	
2	-0.646941	-0.079100	0.009899	-1.259453	
3	0.200808	0.398764	-1.149617	2.336104	
4	-1.054216	3.930077	0.811588	-1.002994	

	browsing_time_minutes	support_interactions
0	0.404395	0.437317
1	0.773937	0.437317
2	2.344915	-0.395573
3	1.316640	-1.228463
4	-0.272570	-0.395573

PART C: Analysis

1. Compare the outputs obtained from different normalization techniques

Original Data		
	annual_income	avg_monthly_spend
0	100473.211709	121.430565
1	54730.644845	572.552674
2	58268.121079	266.593896
3	64829.795654	691.452358
4	27431.467873	832.664792

Min-Max Normalized Data		
	annual_income	avg_monthly_spend
0	0.184765	0.037557
1	0.098106	0.187021
2	0.104808	0.085652
3	0.117239	0.226414
4	0.046387	0.273200

Z-Score Normalized Data		
	annual_income	avg_monthly_spend
0	1.902940	-1.071471
1	0.359788	1.188752
2	0.479127	-0.344170
3	0.700489	1.784466
4	-0.561165	2.491972

Min-Max Normalization scales all numerical values to a range between 0 and 1. It preserves the original distribution and makes different features easier to compare. Z-Score Normalization transforms the data so that the mean is approximately 0 and the standard deviation is approximately 1. While Min-Max normalization restricts values to a fixed range, Z-Score normalization centers the data around the mean and can produce both positive and negative values.

2. Identify which normalization method is most suitable for the selected domain and justify your answer

For the Retail Customer Segmentation dataset, Z-Score Normalization is more suitable. The dataset contains features such as annual income, monthly spending, and purchase frequency that have different scales and may contain outliers. Z-Score Normalization standardizes the data by centering it around the mean and making the standard deviation equal to 1. This allows machine learning algorithms to treat all features equally and improves the accuracy of customer analysis and segmentation.